

Project TODO

Ten steps, in order — verified against actual repo state on 2026-09-02, not aspirational. Attempt each step yourself first; ask for help when stuck. Don't skip ahead out of order — later steps assume earlier ones are actually settled, not just started.

☐ 1. Finish the decode merge

Migrate `segment_loader.cpp`'s two calls (decodeOrder + decodeFill) onto one decodeCapturedOrder call. Delete decodeFill and the CapturedOrderMatchesSeparateDecodesOnARealLine dual-run test once that lands and passes. decodeOrder stays permanently — the legacy recorder path needs order-only decoding forever.

Mostly wiring; fine to hand off.

Done when: One decode call per order frame in `segment_loader.cpp`, full suite green, decodeFill and the dual-run test are gone.

☐ 2. Close the last two Stage 0 items (plan v4 §10)

(a) Activate the clock/floating-price CI guards in `.github/workflows/ci.yml` — currently a TODO comment, not enforced.

(b) Split `OrderBook::validate()` into structural invariants (run after every mutation) vs. decision-ready checks (run only at declared safe checkpoints) per §12's "two invariant modes."

(b) is a real design call — what exactly counts as decision-ready needs a definition first.

Done when: CI actually fails on a real clock/floating-price violation (prove it with a deliberately-broken test), and `OrderBook` exposes two distinct invariant checks.

☐ 3. Write the event-loop causality ADR — before any engine code

§15 names eight unanswered ordering questions: when does an event become visible; when does the book mutate; when does the strategy see the new view; when does a submitted order enter the latency queue; when can it first fill; when are fees/cash/position/PnL updated; how are equal timestamps ordered; what happens while the book is warming/stale/gapped/resyncing. Same discipline that produced ADR 0013: decide on paper first, so steps 4–7 build against a settled contract instead of discovering it mid-implementation.

Learning-critical — wrestle with this one.

Done when: The ADR is accepted, even if some answers are "deferred, blocked on evidence" (a legitimate answer — see how `order_subtype` and book health were handled).

☐ 4. Portfolio / accounting type

include/te/engine/portfolio.hpp's TODO: cash, position, average price, realized/unrealized PnL, fees — int64 ticks, never double. Test-first: write the hand-calculated fill/fee/PnL scenario before the implementation.

Done when: Passes hand-calculated scenarios in isolation, no engine loop required.

☐ 5. ExecutionVenue interface + the simplest possible simulated venue

Just submit, fill, cancel — no latency model yet.

Done when: A submit/fill/cancel round trip is unit-testable without the engine loop.

☐ 6. Strategy interface + NoopStrategy

The callback interface plus the implementation that trades nothing. This is what step 8's first gate runs against.

Done when: NoopStrategy compiles against the real interface and its callbacks are directly testable even before the engine loop exists to drive them.

☐ 7. The engine loop

Wires Feed / Clock / Strategy / ExecutionVenue / OrderBook / Portfolio together, implementing exactly the ordering decided in step 3.

The hard one — pair on this rather than solo it; too much rides on getting the causality right.

Done when: Compiles and runs end-to-end on a trivial synthetic feed.

☐ 8. The three Stage 5 gate tests

- (a) No-op strategy conserves event counts, cash, position, and PnL exactly over a real capture.
- (b) One hand-calculated order/fill/fee/PnL scenario matches exactly.
- (c) Ten identical runs produce identical event/book/order/cash/position/PnL hashes (reuse Replay's digest pattern).

Done when: All three pass — this is what actually closes Stage 5, not elapsed time spent.

☐ 9. Wire up apps/replay_main.cpp

Currently a 7-line stub, commented out of the CMake build.

Done when: cmake --build produces a replay binary that runs a real capture through NoopStrategy end to end and prints a report.

☐ 10. Refresh the docs

Plan v4's stage table, status.md, and the six ghost-header labels touched in the 2026-09-02 session (engine.hpp, portfolio.hpp, strategy.hpp, venue.hpp, simulated_venue.hpp, noop_strategy.cpp) all currently say “reserved, not implemented.” Once Stage 5 closes, that stops being true.

Done when: None of those files describe themselves as unimplemented anymore.

See CLAUDE.md for the working agreement this list follows. Refresh this checklist when a step closes or when reality disagrees with it — a stale checklist is worse than none.